26F Stat TA Session W4

1. Generate Random Numbers

- After 3 weeks of basic data manipulation and plot-drawing (data visualization), you may expect that we are still going to draw figures today. But Hey, this is a probability and statistics course!
- We (should) have learned in the lecture the concept of **random variables** and **distributions**:

> 💡 **Random Variable:**
> A function that assigns a value to each outcome in the sample space
> **Distribution:**
> A description of how probabilities are assigned to the values of a random variable

- **Example:** Roll a Die
  - Let X be the r.v. that equals the number of dots ($\in \{1, 2, 3, 4, 5, 6\}$) on the upper face
  - Each value has probability $1/6$

Random Seed

- Now we are going to draw numbers from certain distributions in `R` , but how can we allow others to reproduce our results if we are drawing randomly?
- A seed allows us to do so by specifying a sampling function: `runif()` , `rnorm()` , `sample()`

```
set.seed(20261001)
rnorm(1, mean = 0, sd = 1)

#output
[1] -0.6833539   # the output should always be the same!
```

Uniform Random Samples `runif()`

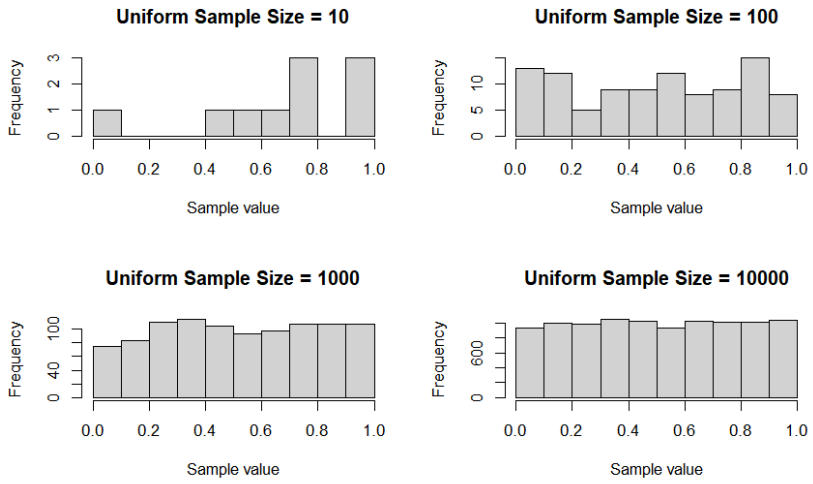


Figure: Random Samples drawn from Uniform[0,1] in different sizes

```r
rdnum <- runif(10)  # default: min = 0, max = 1
rdnum_2 <- runif(n = 10, min = 0, max = 100)

# output
print(rdnum)
[1] 0.60948729 0.12423338 0.84172306 0.87140030 0.21421948
[6] 0.63042043 0.94701880 0.83829748 0.20671720 0.06368964

print(rdnum_2)
[1] 80.18057 92.06480 94.93640 92.88272 96.57980
[6] 29.93912 72.90744 39.00587 16.61714 22.60201
```

2. Coin Toss

❓ **Q.** How can we simulate a coin toss game in `R` ?

- What do we have in coin toss?
  - A fair coin with equal probability of being head or tail
- A uniform distribution on $[0, 1]$ can be divided into equal halves!

```r
head <- rdnum > 0.5
class(head)  # "logical"

# output
```

```
[1] TRUE FALSE TRUE TRUE FALSE
[6] TRUE TRUE TRUE FALSE FALSE
```

- `head` gives `TRUE` or `FALSE` . If we want to do calculations, we have to convert them into numbers!

```
# head gives TRUE or FALSE
# to do calculations, we want them to be numbers
head_numeric <- as.numeric(head)

# output
[1] 1 0 1 1 0 1 1 1 0 0
```

- `as.numeric()` convert things to numbers. Similarly, we have `as.logical()` , `as.integer()` , `as.character()`
- With numeric values, we can calculate the mean and it's more useful for data analysis!

```
mean(head_numeric) # 0.6

rdnum_3 <- runif(10000)
head_numeric_2 <- as.numeric(rdnum_3 > 0.5)
mean(head_numeric_2) # 0.4995 -> closer to 0.5! why?
```

Cumulative Sum of Sequences `cumsum()`

- With an indicator of being head or not, we can count the cumulative number of heads using `cumsum()`

```
cum_head <- cumsum(head_numeric)

# output
[1] 1 1 2 3 3 4 5 6 6 6   # we have 6 heads in 10 tosses
```

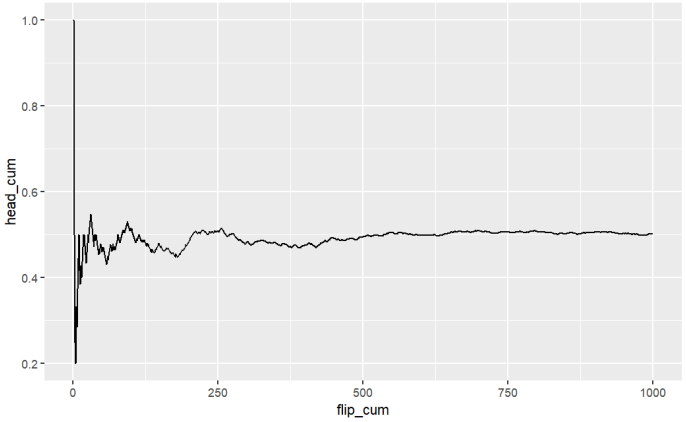


Figure: Head Count at Each Round of the Flip

3. Operators: `$` , `[]` , `[[]]`

- For data cleaning, we may find undesirable data types. But now we can easily convert them into the data type we want!

```
data_example <- data.frame(name = c("Alice", "Bob", "Cindy"),
                           score = c("80", "60", "100")
                           )

mean(data_example$score) # NA!
mean(as.numeric(data_example$score)) # 80
```

- In above example, we use `$` to extract the column we want
- We can also use `[]` or `[[]]`

▼ (Optional) Q. But what's the difference?

```
# [[]] extracts a single element of a list
data_example[["name"]]  # "Alice", 'Bob", "Cindy"
data_example[[1]]

# $ is a shorter version of [[]]
data_example$name  # "Alice", 'Bob", "Cindy"

# [] returns a smaller dataframe
data_example["name"]

# output
   name
1 Alice
2   Bob
3 Cindy
```

Exercises

W4 Exercises